

29 Ekim 2024 Salı 15:53

What is Operating System ?

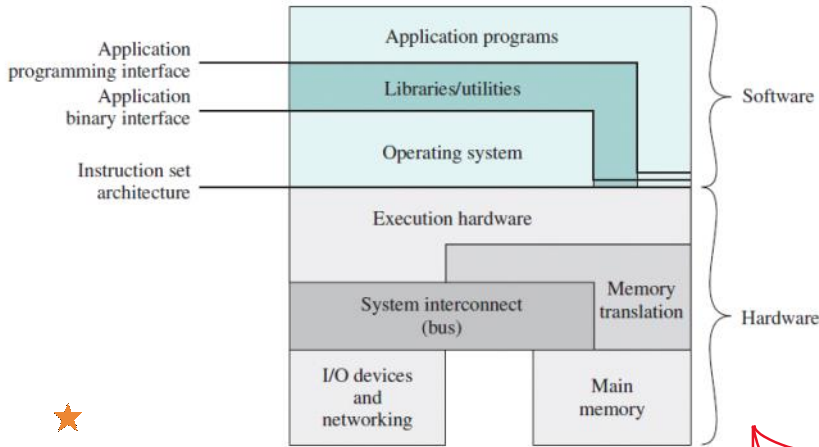
- A program that controls the execution of application programs.
- An interface between applications and hardware. It can be thought of as having three objectives:



Main objectives of an OS:

- Convenience
- Efficiency
- Ability to evolve

- **Convenience:** An OS makes a computer more convenient to use.
- **Efficiency:** An OS allows the computer system resources to be used in an efficient manner.
- **Ability to evolve:** An OS should be constructed in such a way as to permit the effective development, testing, and introduction of new system functions without interfering with service.



Computer systems are organized in layers, with end users only interacting with applications, unaware of hardware details. Application developers rely on system programs and utilities, which simplify tasks like file management and device control. The OS is the main system program, hiding hardware complexity and providing an easy interface for programmers to access system resources. This layered structure reduces development complexity.

Figure 2.1 Computer Hardware and Software Structure

Operating System Services

- Program development
- Program execution
- Access I/O devices
- Controlled access to files
- System access
- Error detection and response
- Accounting



Kullanıcıya uygulama sağlamak için kullanılan **hardware** ve **software**, Şekil 2.1'de gösterildiği gibi katmanlı bir yapıda görülebilir. Bu uygulamaların kullanıcısı (son kullanıcı) genellikle bilgisayar **hardware** ayrıntılarıyla ilgilenmez. Bu nedenle, son kullanıcı bir bilgisayar sistemini bir dizi **application** olarak görür. Bir **application**, bir **programming language** ile ifade edilebilir ve bir **application programmer** tarafından geliştirilir. Eğer bir **application program** tamamen **hardware** kontrolünü üstlenen bir **machine instructions** seti olarak geliştirilmek istenseydi, oldukça karmaşık bir görev olurdu. Bu görevi kolaylaştırmak için bir dizi **system programs** sağlanır. Bu programlardan bazıları **utilities** ya da **library programs** olarak adlandırılır. Bunlar, program oluşturma, dosya yönetimi ve I/O aygıtlarının kontrolü gibi sık kullanılan işlevleri sağlar. Bir programcı, bir **application** geliştirirken bu olanaklardan yararlanır ve **application** çalışırken belirli işlevleri gerçekleştirmek için bu **utilities** programlarını çağırır. En önemli **system programs** koleksiyonunu **OS** oluşturur. **OS**, programcıdan **hardware** ayrıntılarını gizler ve sistemin kullanımı için programcıya uygun bir arayüz sağlar. Bir arabulucu olarak hareket eder, programcı ve **application programs** için bu olanaklara ve hizmetlere erişimi kolaylaştırır.

Operating System Services

Program development: The OS provides a variety of facilities and services, such as editors and debuggers, to assist the programmer in creating programs.

Program execution: A number of steps need to be performed to execute a program. Instructions and data must be loaded into main memory, I/O devices and files must be initialized, and other resources must be prepared.

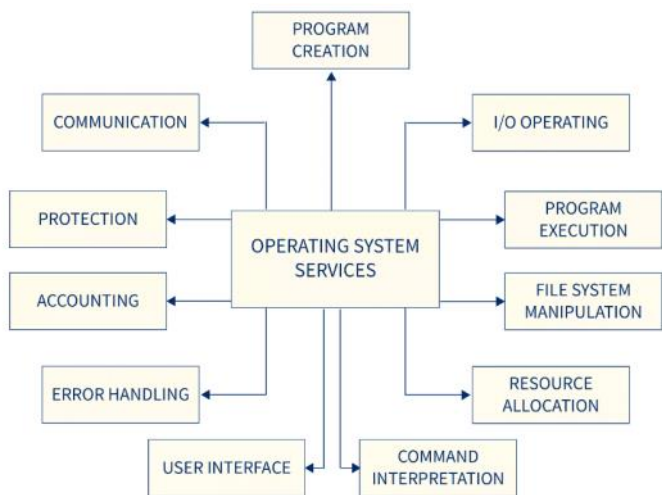
Access to I/O devices: Each I/O device requires its own peculiar set of instructions or control signals for operation.

Controlled access to files: For file access, the OS must reflect a detailed understanding of not only the nature of the I/O device (disk drive, tape drive) but also the structure of the data contained in the files on the storage medium.

System access: For shared or public systems, the OS controls access to the system as a whole and to specific system resources.

Error detection and response: A variety of errors can occur while a computer system is running. These include internal and external hardware errors, such as a memory error, or a device failure or malfunction; and various software errors, such as division by zero, attempt to access forbidden memory location. the OS must provide a response that clears the error condition with the least impact on running applications.

Accounting: A good OS will collect usage statistics for various resources and monitor performance parameters such as response time. On any system, this information is useful in anticipating the need for future enhancements and in tuning the system to improve performance. On a multiuser system, the information can be used for billing purposes.



three key interfaces in a typical computer system:

- Instruction set architecture (ISA)
- Application binary interface (ABI)
- Application programming interface (API)

- 1- **The Instruction Set Architecture (ISA) defines the set of machine language instructions a computer can execute. It serves as the boundary between hardware and software.** Both application and utility programs can access the ISA directly, but they have access only to a subset known as the user ISA. The operating system has access to additional instructions, referred to as the system ISA, which are used for managing system resources.
- 2- **Application binary interface (ABI) : The ABI defines a standard for binary portability across programs.** The ABI defines the system call interface to the operating system and the hardware resources and services available in a system through the user ISA.
- 3- **Application programming interface (API): The API gives a program access to the hardware resources and services available in a system through the user ISA supplemented with high-level language (HLL) library calls.** Any system calls are usually performed through libraries. Using an API enables application software to be ported easily, through recompilation, to other systems that support the same API.

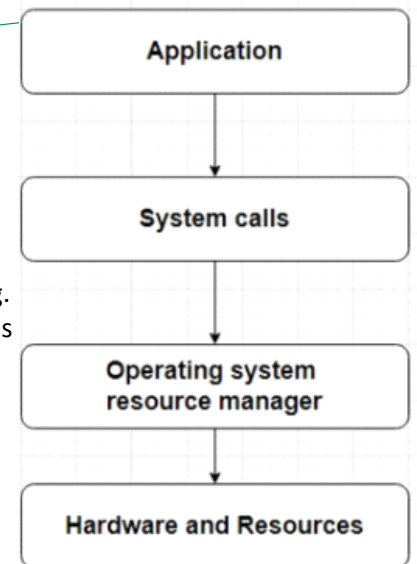
The Operating System as Resource Manager

- **The OS is responsible for controlling the use of a computer's resources, such as I/O, main and secondary memory, and processor execution time.**

The OS, as a control mechanism is unusual in two respects:

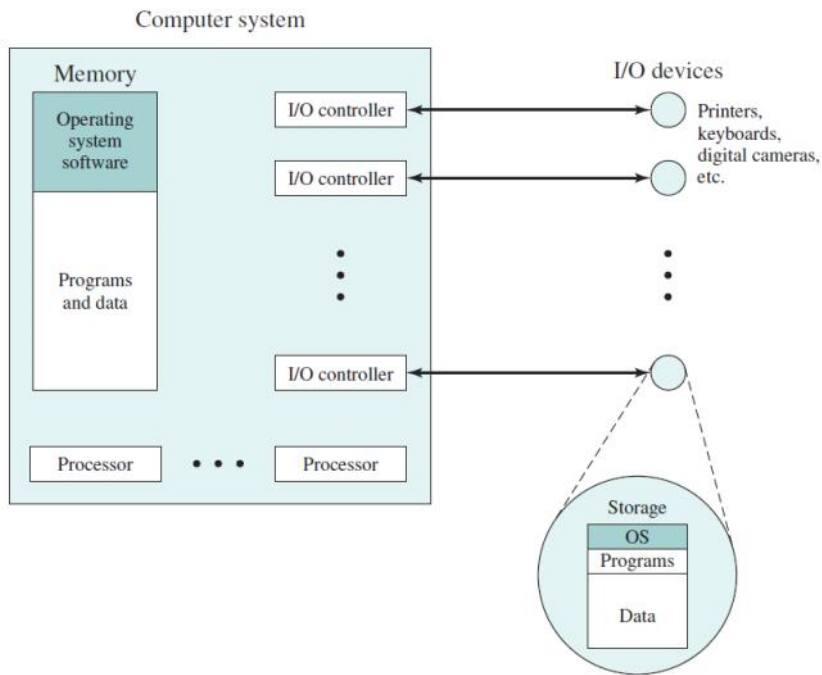
- **The OS functions in the same way as ordinary computer software;** that is, it is a **program or suite of programs executed by the processor.**
- **The OS frequently relinquishes control** and must depend on the processor to allow it to regain control.

The OS gives instructions to the processor and manages resource usage and execution timing. To allow the processor to perform tasks, it temporarily relinquishes control to other programs and then regains it to prepare for the next task.



Resource management in Operating System is the process to manage all the resources efficiently like CPU, memory, input/output devices, and other hardware resources among the various programs and processes running in the computer.

Resource management is an important thing because resources of a computer are limited and multiple processes or users may require access to the same resources like CPU, memory etc. at the same time. The operating system has to manage and ensure that all processes get the resources they need to execute, without any problems like deadlocks.



The processor itself is a resource, and the **OS must determine** how much processor time is to be devoted to the execution of a particular user program.

Figure 2.2 The Operating System as Resource Manager

Ease of Evolution of an Operating System

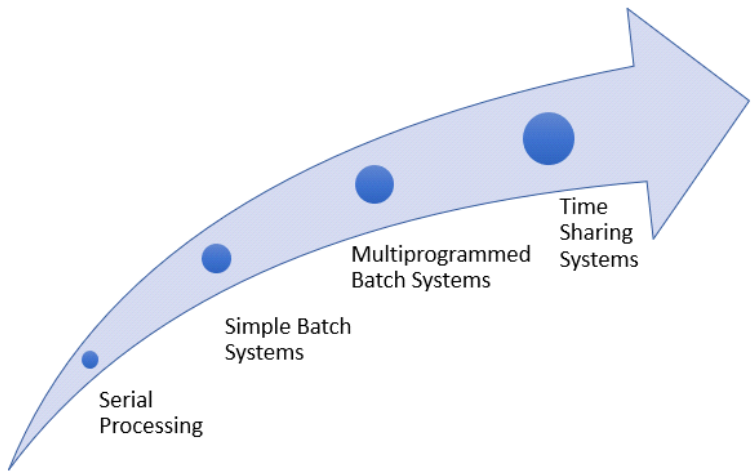
OS will evolve over time for a number of reasons;

Hardware upgrades + new types of hardware

New services: In response to user demand or in response to the needs of system managers, the OS expands to offer new services.

Fixes: Any OS has faults. These are discovered over the course of time and fixes are made.

Evolution of Operating Systems



1- Serial Processing

- No operating system.
- User interacted directly with the computer hardware
- Computers ran from a console with display lights.
- Users have access to the computer in “series”

Two Main Problems

Scheduling : Most installations used a **hardcopy sign-up sheet to reserve computer time**. (Time allocations could run long/ short ? = end of waste of time)

Setup time : A considerable amount of time was spent on setting up the program to run. (called job - funct- main memory - load- link...)

2- Simple Batch System

- Early computers were very expensive, and therefore it was important to maximize processor utilization.
- Monitor
 - With this type of OS, the user no longer has direct access to processor.
 - Job is submitted to computer operator who batches them together and places them on an input device.
 - Program branches back to the monitor when finished.

View of Monitor Point

- **Monitor controls the sequence of events.** For this to be so, much of the monitor must always be in main memory and available for execution. (This section is called as resident memory.)
- **Resident Monitor** is (software) always in memory.
- Monitor reads in jobs one at a time from the input device and gives to the control.
- When the Job is completed, it returns control to the monitor. which immediately reads the next job.

View of Processor Point

- **Processor executes instruction from the memory** containing the monitor.
- Executes the instructions in the user program until it encounters an ending or error condition.
- **“Control is passed to a job”** means processor is fetching and executing instructions in a user program.
- **“Control is returned to the monitor”** means that the processor is fetching and executing instructions from the monitor program.

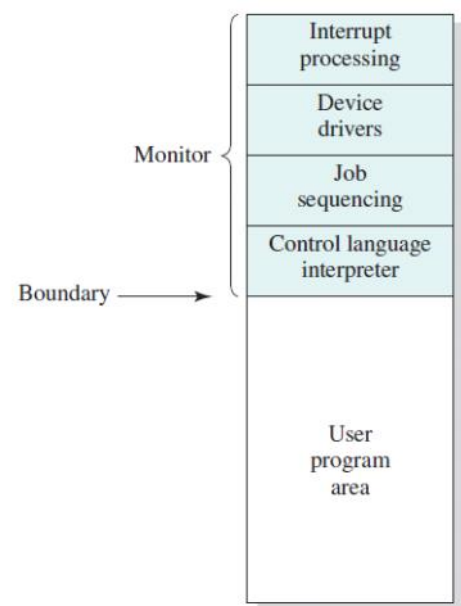


Figure 2.3 Memory Layout for a Resident Monitor

JCL is a special type of programming language used to provide instructions to the monitor



What compiler to use



What data to use

The monitor handles scheduling by organizing jobs in a line (or queue) and running them one after another without any breaks in between. This way, the monitor reduces the time needed to set up each job. For each job, there are also simple instructions included in a basic job control language (JCL).

Desirable Hardware Features

Memory protection

- **While the user program is executing, it must not alter the memory area containing the monitor.** If such an attempt is made, the processor hardware should detect an error and transfer control to the monitor. The monitor would then abort the job, print out an error message, and load in the next job.

Timer

- **Timer is used to prevent a job from monopolizing the system.** The timer is set at the beginning of each job. If the timer expires, the user program is stopped, and control returns to the monitor.

Privileged instructions

- **Certain machine level instructions are designated privileged** and can only be executed by the monitor.

Interrupts

- **Gives OS more flexibility** in controlling user programs(relinquishing control & regaining control from user programs.)

Modes of Operation

User Mode

- User program executes in user mode
- Certain areas of memory are protected from user access
- Certain instructions may not be executed

Kernel Mode

- Monitor executes in kernel mode.
- Privileged instructions may be executed.
- Protected areas of memory may be accessed.

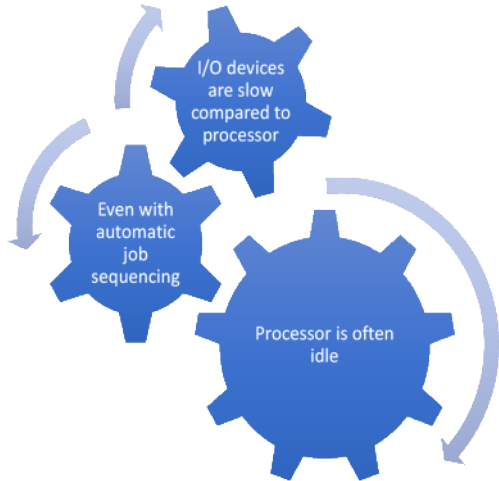


Simple Batch System Overhead(Ek Yük)

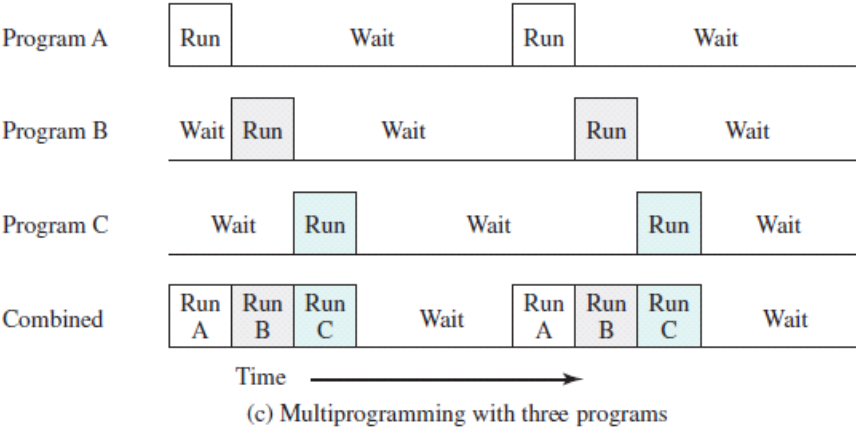
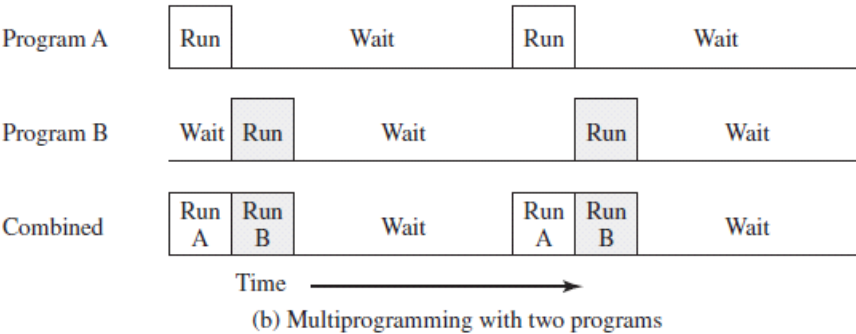
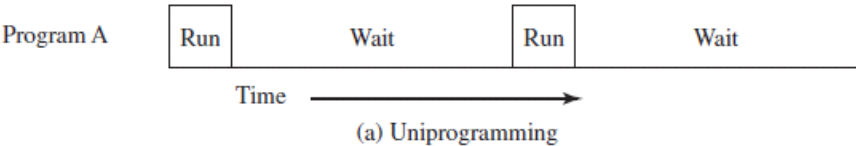
- With a batch OS, processor time alternates between execution of user programs and execution of the monitor.
- There've been two **sacrifices**:
 - **Some main memory is now given over to the monitor.**
 - **Some processor time is consumed by the monitor.**
- Despite these overhead, the **simple batch system improves utilization of the computer.**



3- Multiprogrammed Batch Systems



Even with the automatic job sequencing provided by a simple batch OS, the processor is often idle. The problem is that I/O devices are slow compared to the processor.



4- Time Sharing System

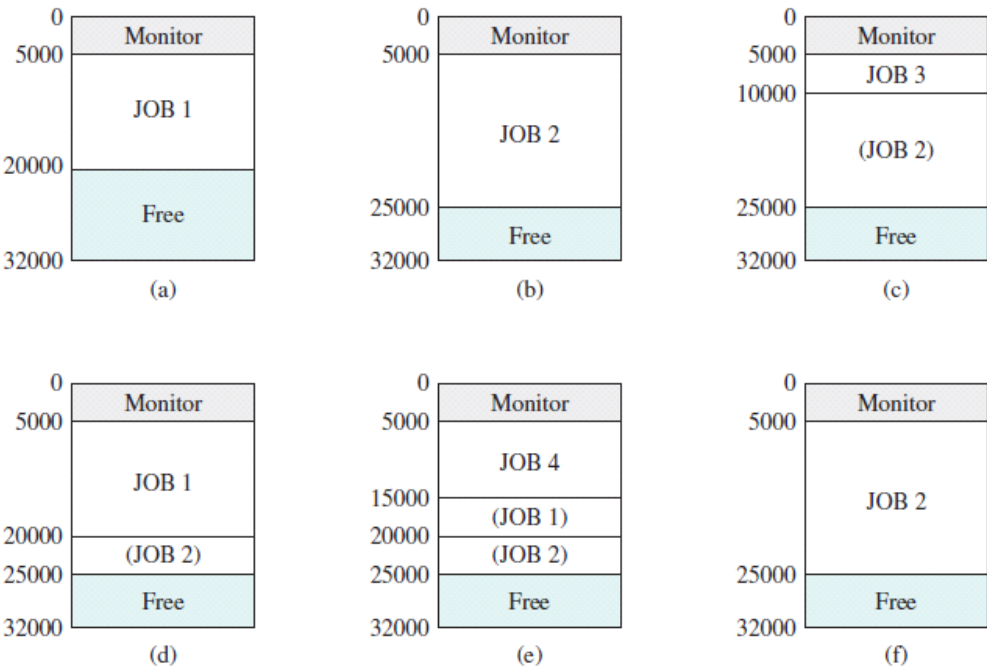
- Just as multiprogramming allows the processor to handle multiple batch jobs at a time, **multiprogramming can also be used to handle multiple interactive jobs.** The technique is referred to as **time sharing**, **because processor time is shared among multiple users.**
- Multiple users simultaneously access the system through terminals, with the OS interleaving the execution of each user program in a short burst or quantum of computation
- If there are **n** users all using the computer at the same time, **each user will get, on average, $1/n$ of the computer's total power.** This does not include the resources used by the OS.

	Batch Multiprogramming	Time Sharing
Principal objective	Maximize processor use	Minimize response time
Source of directives to operating system	Job control language commands provided with the job	Commands entered at the terminal

Both batch processing and time sharing use multiprogramming. The key differences are listed in the table.

Compatible Time-Sharing System (CTSS)

- Utilized a technique known as **time slicing**
 - System clock generated interrupts at a rate of approximately one every 0.2 second
 - At each clock interrupt the OS regained control and could assign the processor to another user
 - Thus, at regular time intervals the current user would be preempted and another user loaded in
 - To preserve the old user program status for later resumption, the old user programs and data were written out to disk before the new user programs and data were read in
 - Old user program code and data were restored in main memory when that program was next given a turn
- One of the first time-sharing operating systems**
- Developed at MIT by a group known as Project MAC
- The system was first developed for the IBM 709 in 1961
- Ran on a computer with 32,000 36-bit words of main memory, with the resident monitor consuming 5000 of that



To minimize disk traffic, user memory was only written out when the incoming program would overwrite it. This principle is illustrated in Figure 2.7 .

- JOB1: 15,000
- JOB2: 20,000
- JOB3: 5000
- JOB4: 10,000

Figure 2.7 CTSS Operation

First, the monitor loads JOB1 and starts it. Later, it switches to JOB2, but since JOB2 needs more memory, JOB1 is saved to free up space. Then, JOB3 is loaded; it's smaller, so part of JOB2 can stay in memory, saving time. When the monitor returns(control) to JOB1, another part of JOB2 is saved. When JOB4 is loaded, parts of JOB1 and JOB2 are kept. If JOB1 or JOB2 is used again, only part of them needs to load back. Finally, JOB2 runs, so JOB4 and the rest of JOB1 are saved to load the missing part of JOB2.

Major Achievements

- Operating Systems are among the most complex pieces of software ever developed.
- Major advances in development include:
 - Processes
 - Memory management
 - Information protection and security
 - Scheduling and resource management
 - System structure

Process

Fundamental to the structure of operating systems

A *process* can be defined as:

A program in execution

An instance of a running program

The entity that can be assigned to, and executed on, a processor

A unit of activity characterized by a single sequential thread of execution, a current state, and an associated set of system resources

Causes of Errors

Deadlocks

- It is possible for two or more programs to be hung up waiting for each other.

Improper synchronization

It is often the case that a routine must be suspended awaiting an event elsewhere in the system. Improper design of the signaling mechanism can result in loss or duplication.

Failed mutual exclusion

More than one user or program attempts to make use of a shared resource at the same time. A mutual exclusion mechanism is needed to allow only one routine at a time to update the file.

Nondeterminate program operation

- When programs share memory and the processor switches between them, they might accidentally change the same memory areas, causing unexpected problems.
- The order in which programs are scheduled may affect the outcome of any particular program

Components of a Process

- A process contains three components:
 - An **executable program**
 - The associated data needed by the program (variables, work space, buffers, etc.)
 - The **execution context** (or "process state") of the program
- The execution context is essential:
 - The execution context is the internal data by which the **OS is able to supervise and control the process.**
 - Includes the contents of the various process registers, PC, data registers.
 - Includes information such as the priority of the process and whether the process is waiting for the completion of a particular I/O event

Process Management

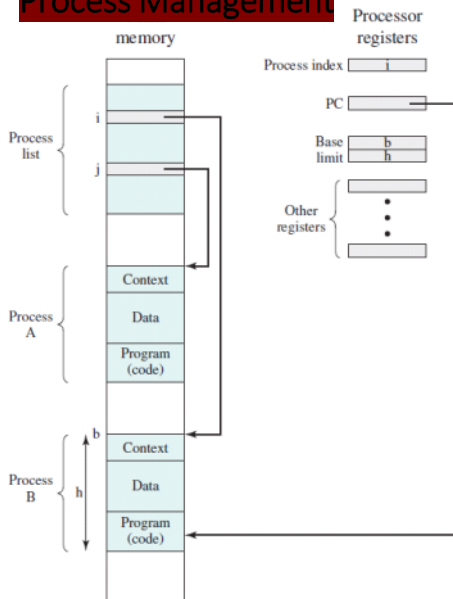


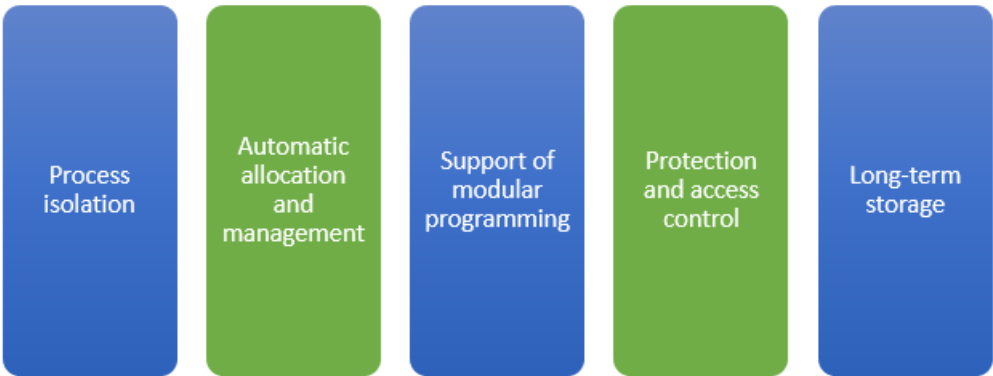
Figure 2.8 Typical Process Implementation

Figure 2.8 shows how processes A and B are managed in memory. Each process has a block of memory with its program, data, and context info. The OS keeps a list of processes, with each entry pointing to where each process is stored in memory. The process index register shows which process is currently using the processor, and the program counter points to the next instruction.

In this figure, process B is running, while process A was paused. When A was stopped, all its info was saved so the OS can switch back to it later. To resume A, the OS saves B's context, restores A's, and sets the program counter to A's location so it continues running where it left off.

Memory Management

The OS has five principal storage management responsibilities:



Automatic allocation and management: Programs should be dynamically allocated across the memory hierarchy as required. Therefore, the programmer does not having to worry about memory limits. Thhis way, the OS manages memory efficiently, giving memory to programs only when necessary.

Protection and access control: The OS must control memory access to protect programs and itself while allowing **safe sharing** when needed.

Process isolation: The OS must prevent independent processes from interfering with each other’s memory, both data and instructions.

Long-Term Storage : Some programs need to store information for a long time, even after the computer is turned off.

Support of modular programming: Programmers should be able to define program modules, and to create, destroy, and alter the size of modules dynamically.

Virtual Memory

A system lets programs use memory in a logical way, ignoring the physical memory size. This allows multiple jobs to stay in memory and run at the same time.

Paging

- Allows processes to be involve of a number of fixed-size blocks, called pages.
- Program references a word by means of a *virtual address*, consisting of a page number and an offset within the page.
- Each page of a process may be located anywhere in main memory.
- The paging system provides for a dynamic mapping between the virtual address used in the program and a *real address* (or physical address) in main memory.

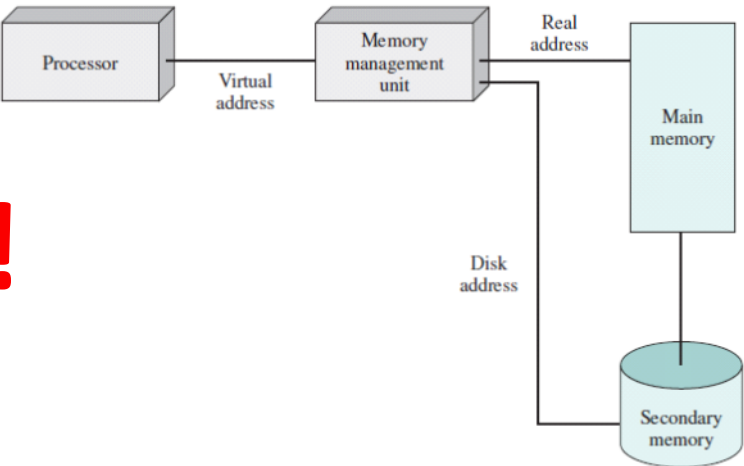
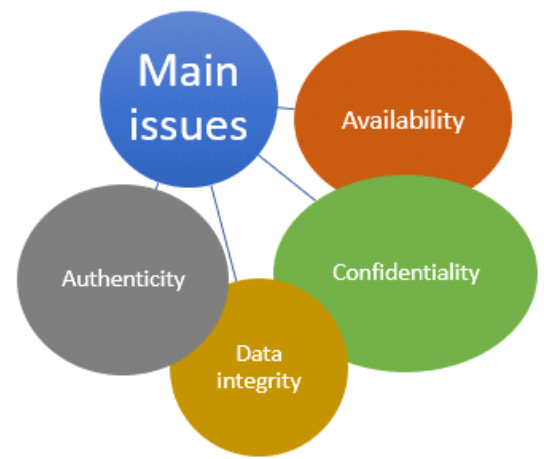


Figure 2.10 shows how addressing works in a virtual memory system. It includes fast main memory and slower auxiliary memory, which is used when needed. The memory management unit helps translate virtual addresses into real memory addresses. If a program tries to access a virtual address not in main memory, some data is moved to auxiliary memory, and the needed data is brought into main memory. While this happens, the program must be paused. The OS must create a system for address translation that has low overhead and manages memory use to reduce movement between memory types.

Figure 2.10 Virtual Memory Addressing

Information Protection and Security

- The **problem involves controlling access to computer systems** and the information stored in them



- **Availability:** Concerned with protecting the system against interruption.
- **Confidentiality:** Assures that users cannot read data for which access is unauthorized.
- **Data integrity:** Protection of data from unauthorized modification.
- **Authenticity:** Concerned with the proper verification of the identity of users and the validity of messages or data.

Scheduling and Resource Management

- Key responsibility of an OS is **managing resources**.

Resource allocation policies must consider;

- **Efficiency**
- **Differential responsiveness**
- **Fairness**

Efficiency : The OS should **maximize productivity**, **minimize response time**, and **support many users in time-sharing**, though these goals can collision. **Balancing** them is a key issue in OS research. Scheduling and resource management are operations-research problems, and monitoring system activity is vital for performance adjustments.

Fairness: We want all processes competing for a specific resource to have fair and equal access, especially when the jobs have similar needs.

Differential responsiveness : The OS may need to treat different types of jobs with different needs differently. **It should make decisions about resource allocation and scheduling based on all job requirements** and do this dynamically. For example, if a process is waiting for an I/O device, the OS should schedule it to run quickly so it can use the device and then free it for other processes.

Different Architectural Approaches

Requesting on operating systems require new ways of organizing the OS.

Different approaches and design elements have been tried:

- Microkernel architecture
- Multithreading
- Symmetric multiprocessing
- Distributed operating systems
- Object-oriented design

Microkernel Architecture

Microkernel architecture assigns only a few essential functions to the kernel:



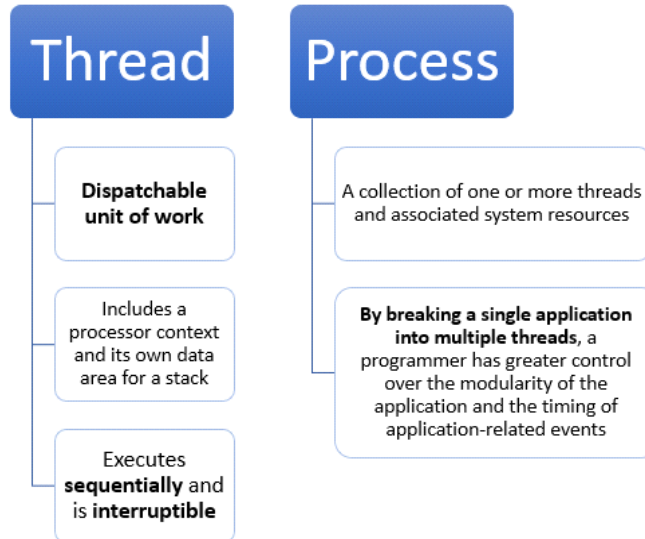
• The approach:



Most operating systems used to have a large monolithic kernel that included many functions like scheduling, file systems, and memory management, all running in one address space. In contrast, a **microkernel has only essential functions like address spaces, interprocess communication (IPC) and basic scheduling**. Other services run in user mode as separate processes called servers. This separation allows for **easier development** and customization of servers for specific needs. The microkernel design is **simpler, flexible, and works well in distributed systems**.

Multithreading

- Multithreading is a technique in which a process, executing an application, is divided into threads that can run concurrently.



- Thread: A **thread is a small unit of work** with its own context (program counter and stack pointer) and data area. It runs sequentially and can be interrupted to allow other threads to run.
- **Process**: A process is a **group of one or more threads** that share resources like memory and files. It represents a **running program**, allowing better control over tasks through multiple threads.

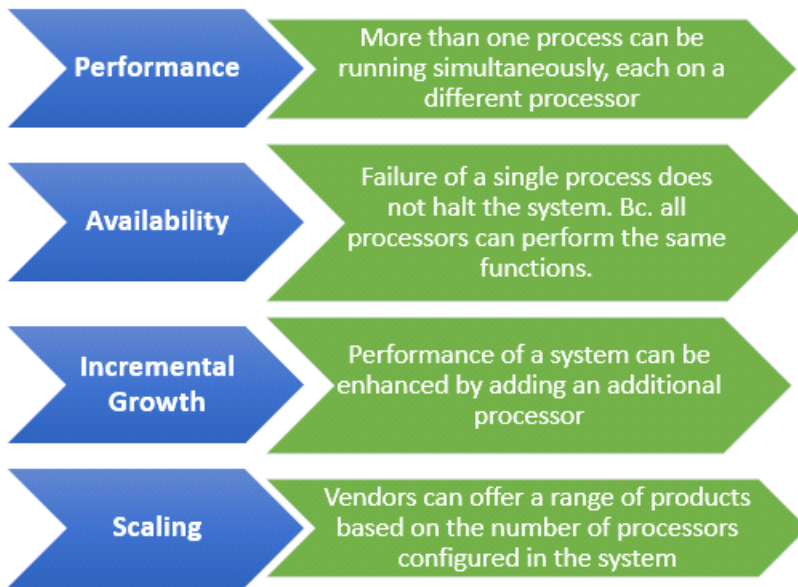
Multithreading is useful for applications that perform a number of essentially independent tasks that do not need to be serialized.

Symmetric Multiprocessing (SMP)

- Term that refers to a computer hardware architecture and also to the OS behavior that exploits that architecture.
- **The OS of an SMP schedules processes or threads across all of the processors.**
- The OS must provide tools and functions to exploit the **parallelism** in an SMP system.
- **Multithreading** and **SMP** are often discussed together, but the two are independent facilities.
- An attractive feature of an **SMP is that the existence of multiple processors is transparent to the user**, the user can use the system without being aware of the multiplicity of processors

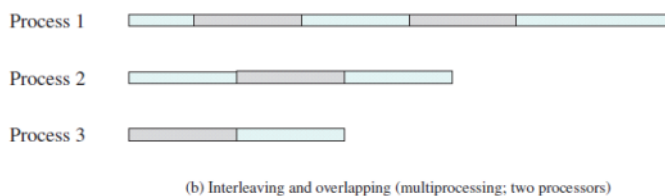
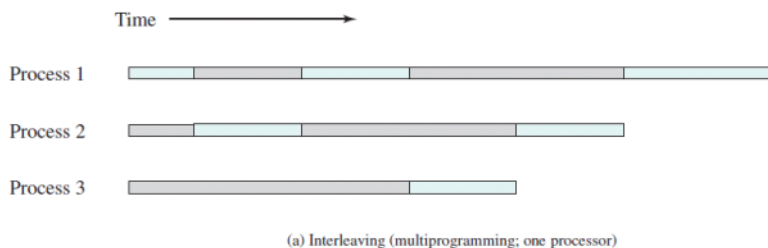


SMP Advantages



It is important to note that these are **potential**, rather than guaranteed, benefits.

The OS must provide tools and functions to take advantage of the parallelism in an SMP system.



Blocked Running

Figure 2.12 Multiprogramming and Multiprocessing

If work can be done in parallel, a system with **multiple processors will perform better than one with a single processor**. In multiprogramming, only one process runs at a time, while other processes wait. In contrast, multiprocessing allows multiple processes to run at the same time on different processors.

Fault Tolerance

- Refers to the ability of a system or component to continue normal operation despite the presence of hardware or software faults. (**despite the errors , ability the continue the normal operations.**)
- Typically involves some degree of redundancy
- Intended to increase the reliability of a system
 - > Typically comes with a cost in financial terms or performance
- The extent adoption of fault tolerance measures must be determined by how critical the resource is.

